Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Дворак В. А

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Вариант 4

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score

# Задаем manual seed = 123 для воспроизводимости результатов
SEED = 123
torch.manual_seed(SEED)

# 1. Импорт библиотек и подготовка данных
digits = load_digits()
X, y = digits.data, digits.target

# Стандартизация признаков
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Разделение на обучающую и тестовую выборки (70/30) с фиксированным seed
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=SEED, stratify=y
)

# Преобразование данных в тензоры PyTorch
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)

# Для многоклассовой классификации (nn.CrossEntropyLoss) метки должны быть
типа int64 (torch.long)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
y_test_tensor = torch.tensor(y_test, dtype=torch.long)

# 2. Определение архитектуры нейронной сети

# Базовая модель: 1 скрытый слой с 32 нейронами (ReLU)
class MLPBase(nn.Module):
    def __init__(self, input_dim):
        super(MLPBase, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 10)  # Выходной слой (10 классов)
        )

    def forward(self, x):
        return self.model(x)

# Экспериментальная модель: 2 скрытых слоя по 32 нейрона (ReLU)
class MLPExp(nn.Module):
    def __init__(self, input_dim):
        super(MLPExp, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
```

```
            nn.ReLU(),
            nn.Linear(32, 10)
        )

    def forward(self, x):
        return self.model(x)

# Функция для обучения и оценки модели
def train_and_evaluate(model, epochs=100, lr=0.001):
    # 3. Инициализация функции потерь и оптимизатора
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    # 4. Цикл обучения (Training Loop)
    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train_tensor)
        loss = criterion(y_pred, y_train_tensor)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 20 == 0:
            print(f"Epoch {epoch + 1}/{epochs}, Loss: {loss.item():.4f}")

    # 5. Оценка модели (Evaluation)
    model.eval()
    with torch.no_grad():
        y_logits = model(X_test_tensor)
        y_pred_class = torch.argmax(y_logits, dim=1)

    acc = accuracy_score(y_test_tensor.numpy(), y_pred_class.numpy())
    f1 = f1_score(y_test_tensor.numpy(), y_pred_class.numpy(),
average='macro')

    return acc, f1

# Запуск базовой модели
print(" 1. Базовая модель (1 скрытый слой: 32) ")
torch.manual_seed(SEED)  # сбрасываем seed перед инициализацией весов
base_model = MLPBase(input_dim=X_train.shape[1])
acc_base, f1_base = train_and_evaluate(base_model, epochs=100)
print(f"Accuracy: {acc_base:.4f}")
print(f"F1-score: {f1_base:.4f}\n")

#  Запуск экспериментальной модели
print(" 2. Экспериментальная модель (2 скрытых слоя: 32-32) ")
torch.manual_seed(SEED)  # сбрасываем seed перед инициализацией весов
exp_model = MLPExp(input_dim=X_train.shape[1])
acc_exp, f1_exp = train_and_evaluate(exp_model, epochs=100)
print(f"Accuracy: {acc_exp:.4f}")
print(f"F1-score: {f1_exp:.4f}\n")

#  Вывод эксперимента
print(" Сравнение результатов ")
print(f"Точность 1-слойного MLP: {acc_base * 100:.2f}%")
print(f"Точность 2-слойного MLP: {acc_exp * 100:.2f}%")
```

```
 1. Базовая модель (1 скрытый слой: 32)
Epoch 20/100, Loss: 1.9990
Epoch 40/100, Loss: 1.6117
Epoch 60/100, Loss: 1.1987
Epoch 80/100, Loss: 0.8445
Epoch 100/100, Loss: 0.5992
Accuracy: 0.9185
F1-score: 0.9179

 2. Экспериментальная модель (2 скрытых слоя: 32-32)
Epoch 20/100, Loss: 2.1093
Epoch 40/100, Loss: 1.7245
Epoch 60/100, Loss: 1.1798
Epoch 80/100, Loss: 0.6902
Epoch 100/100, Loss: 0.3853
Accuracy: 0.9148
F1-score: 0.9133

 Сравнение результатов
Точность 1-слойного MLP: 91.85%
Точность 2-слойного MLP: 91.48%
```

Вывод: построил, обучил и оценил многослойный перцептрон (MLP) для решения задачи классификации